

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



Polar vs Dask

Dữ liệu lớn

Mã số sinh viên	Họ và tên
23120245	NGUYỄN QUANG DUY
23120258	LƯU TRỌNG HIẾU
23120260	VĂN ĐÌNH HIẾU

Môn: Dữ liệu lớn

Thành phố Hồ Chí Minh – 2026

Mục lục

1	Introduction	2
2	Overview of Pandas, Polars, and Dask	2
2.1	Pandas	2
2.2	Polars	2
2.3	Dask	3
3	Open-source Status, License, and Pricing	3
4	Architecture and Core Techniques	4
4.1	Pandas Architecture	4
4.2	Polars Architecture	4
4.3	Dask Architecture	5
5	Dataset and Benchmark Workloads	6
5.1	Dataset Overview	6
5.2	Benchmark Workloads	6
6	Benchmark Methodology	7
6.1	Benchmark Objective	7
6.2	Execution Strategy	7
6.3	Benchmark Groups	8
6.4	Metrics and Interpretation	8
7	Experimental Results and Analysis	8
7.1	Real Data Row Scaling	8
7.2	Physical Scaling and Memory Pressure	9
7.3	Synthetic Stress Benchmark	10
7.4	Environment Comparison	12
7.5	Polars Lazy vs Eager	13
8	Multi-aspect Comparison	13
9	Feature Demonstration and Scenario-based Comparison	14
10	Limitations	15
11	Conclusion and Recommendations	16
	References	16

1 Introduction

Trong các hệ thống phân tích dữ liệu hiện đại, *DataFrame* là một mô hình dữ liệu quan trọng để xử lý dữ liệu dạng bảng. Trong hệ sinh thái Python, Pandas là công cụ quen thuộc nhất cho dạng xử lý này, nhưng khi dữ liệu tăng lên quy mô lớn, Pandas bắt đầu gặp hạn chế về hiệu năng, bộ nhớ và khả năng mở rộng. Vì vậy, các công cụ như Polars và Dask được sử dụng để giải quyết các bài toán dữ liệu lớn hiệu quả hơn.

Polars tập trung vào hiệu năng cao trên một máy đơn thông qua columnar execution, Rust engine, multi-threading và query optimization. Trong khi đó, Dask tập trung vào khả năng mở rộng bằng cách chia dữ liệu thành nhiều partition, xây dựng task graph và thực thi song song hoặc phân tán trên nhiều worker.

Báo cáo này tập trung tìm hiểu và so sánh Polars và Dask trong bối cảnh xử lý dữ liệu lớn bằng Python. Pandas được giới thiệu như một công cụ nền tảng và baseline, giúp làm rõ vì sao cần các framework mới hơn. Nội dung chính của báo cáo gồm: tổng quan mục đích thiết kế và mức độ phổ biến của từng công cụ, trạng thái mã nguồn mở và chính sách giá, kiến trúc tổng quát, các kỹ thuật đặc thù, cũng như so sánh thực nghiệm thông qua các workload phổ biến như filter, groupby, join và pipeline.

2 Overview of Pandas, Polars, and Dask

2.1 Pandas

Pandas là thư viện DataFrame phổ biến nhất trong Python, thường được dùng cho data cleaning, exploratory data analysis và prototyping. Pandas phù hợp với dữ liệu nhỏ đến vừa, nơi toàn bộ dataset có thể được nạp vào bộ nhớ chính. Nhờ API đơn giản, tài liệu phong phú và khả năng tích hợp với NumPy, Matplotlib, Scikit-learn và Jupyter Notebook, Pandas có mức độ phổ biến rất cao trong cộng đồng data science.

Tuy nhiên, Pandas không được thiết kế như một Big Data framework. Khi dữ liệu tăng lên hàng chục triệu dòng hoặc vượt quá khả năng bộ nhớ, Pandas thường gặp hạn chế về runtime và memory usage. Vì vậy, trong báo cáo này, Pandas chủ yếu đóng vai trò baseline để làm rõ động lực sử dụng Polars và Dask.

2.2 Polars

Polars là một DataFrame engine hiện đại, được thiết kế để xử lý dữ liệu dạng bảng với hiệu năng cao trên một máy đơn. Polars được viết bằng Rust, sử dụng columnar memory layout và hỗ trợ multi-threaded execution. Công cụ này phù hợp với các workload phân tích dữ liệu lớn nhưng vẫn muốn chạy hiệu quả trên single machine.

Trong cộng đồng Big Data và data engineering, Polars ngày càng phổ biến vì có tốc độ

xử lý nhanh, API dễ tiếp cận và hỗ trợ cả eager execution lẫn lazy execution. Polars đặc biệt phù hợp với các tác vụ filter, projection, aggregation và multi-step pipeline, nhất là khi dữ liệu được lưu ở định dạng columnar như Parquet.

2.3 Dask

Dask là một framework Python dùng cho parallel computing và distributed computing. Với Dask DataFrame, dữ liệu được chia thành nhiều partition, cho phép xử lý song song trên nhiều core hoặc nhiều worker. API của Dask DataFrame khá gần với Pandas, nên người dùng Pandas có thể chuyển sang Dask tương đối dễ dàng khi cần xử lý dữ liệu lớn hơn.

Trong cộng đồng Big Data, Dask phổ biến vì giúp mở rộng các workload Python quen thuộc sang môi trường song song hoặc phân tán. Dask phù hợp với dữ liệu lớn hơn bộ nhớ của một máy, các pipeline có thể chia partition, hoặc các bài toán cần scale từ local machine lên cluster. Tuy nhiên, Dask cũng có overhead từ scheduler, task graph và shuffle, nên không phải lúc nào cũng nhanh hơn các engine single-machine như Polars.

3 Open-source Status, License, and Pricing

Cả Pandas, Polars và Dask đều là các công cụ mã nguồn mở. Người dùng có thể sử dụng miễn phí trong học tập, nghiên cứu và triển khai thực tế. Tuy nhiên, chi phí thực tế khi xử lý Big Data không chỉ đến từ phần mềm, mà còn đến từ tài nguyên phần cứng, cloud, storage hoặc cluster.

Bảng 1: Open-source status, license, and pricing

Tool	Open-source	License	Pricing model
Pandas	Yes	BSD 3-Clause	Free to use. Chi phí chủ yếu đến từ máy local hoặc cloud dùng để chạy workload.
Polars	Yes	MIT License	Core library miễn phí. Có thể phát sinh chi phí nếu dùng cloud infrastructure hoặc dịch vụ thương mại như Polars Cloud.
Dask	Yes	BSD 3-Clause	Dask open-source miễn phí. Chi phí phụ thuộc vào local machine, cloud VM hoặc distributed cluster.

Pandas và Dask sử dụng BSD 3-Clause license, còn Polars sử dụng MIT License. Đây đều là các giấy phép mã nguồn mở permissive, cho phép sử dụng trong nhiều bối cảnh khác nhau, bao gồm cả nghiên cứu và thương mại. Pandas được cấp phép theo BSD 3-Clause, Dask cũng dùng BSD 3-Clause, còn Polars được phân phối theo MIT License. [\[2, 6, 4\]](#)

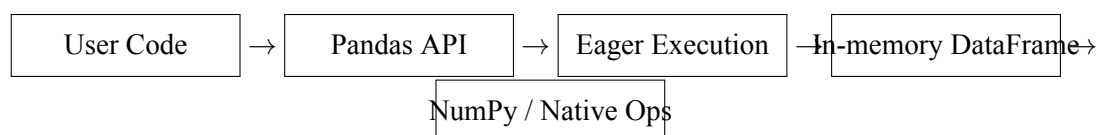
Về pricing, bản thân các thư viện đều miễn phí. Sự khác biệt nằm ở chi phí vận hành. Pandas thường chỉ cần một máy đơn, nhưng khi dữ liệu lớn có thể cần máy có RAM cao. Polars có thể giảm chi phí xử lý nhờ runtime nhanh trên single machine. Dask có thể phát sinh chi phí cao hơn nếu triển khai trên cluster, nhưng đổi lại có khả năng xử lý workload phân tán tốt hơn.

4 Architecture and Core Techniques

Mặc dù Pandas, Polars và Dask đều cung cấp DataFrame API, kiến trúc bên trong của chúng khác nhau đáng kể. Pandas ưu tiên sự đơn giản và thao tác in-memory trên một máy đơn. Polars ưu tiên hiệu năng cao cho single-machine analytics thông qua columnar execution và query optimization. Dask lại tập trung vào khả năng mở rộng bằng cách chia nhỏ dữ liệu thành các partition và thực thi computation thông qua task scheduling.

4.1 Pandas Architecture

Pandas sử dụng kiến trúc single-machine in-memory. Dữ liệu thường được nạp vào RAM dưới dạng DataFrame, sau đó các thao tác như filter, groupby hoặc join được thực thi trực tiếp. Pandas dùng *eager execution*, nghĩa là mỗi operation thường được chạy ngay khi được gọi. Cách thiết kế này giúp Pandas dễ hiểu, dễ debug và phù hợp với exploratory data analysis hoặc các workload nhỏ đến vừa.

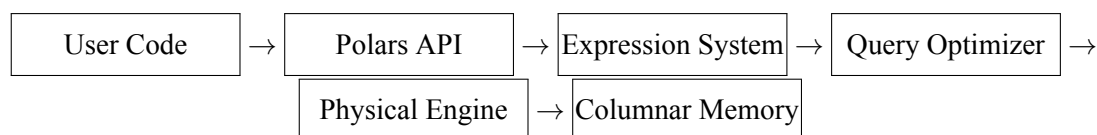


Hình 1: Simplified Pandas architecture

Kỹ thuật chính của Pandas là sử dụng *vectorized operations* thông qua NumPy hoặc native code để tăng tốc xử lý so với vòng lặp Python thuần. Tuy nhiên, Pandas không có query optimizer tổng thể cho toàn bộ pipeline. Vì vậy, khi người dùng thực hiện nhiều bước liên tiếp, Pandas có thể tạo ra nhiều intermediate DataFrame và làm tăng memory usage. Đây là lý do Pandas thường gặp hạn chế khi dữ liệu lớn hơn RAM hoặc khi pipeline tạo ra output trung gian quá lớn.

4.2 Polars Architecture

Polars được thiết kế như một columnar DataFrame engine cho single-machine analytics. Thay vì xử lý dữ liệu theo hàng, Polars tổ chức và xử lý dữ liệu theo cột. Cách tiếp cận này phù hợp với workload phân tích vì nhiều truy vấn chỉ sử dụng một số column nhất định. Engine của Polars được viết bằng Rust, hỗ trợ multi-threaded execution và có thể tận dụng tốt nhiều CPU cores trên cùng một máy.



Hình 2: Simplified Polars architecture

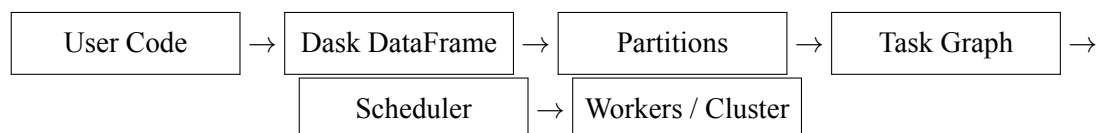
Một đặc điểm quan trọng của Polars là hỗ trợ cả *eager execution* và *lazy execution*. Với eager execution, operation được thực hiện ngay, phù hợp với các thao tác đơn giản hoặc khi cần kiểm tra kết quả từng bước. Với lazy execution, Polars chưa chạy ngay mà xây dựng một query plan cho toàn bộ pipeline. Sau đó, optimizer có thể tối ưu plan trước khi chuyển sang physical execution.

Các kỹ thuật đặc thù của Polars gồm *columnar execution*, *predicate pushdown*, *projection pushdown*, query optimization và parallel execution. Trong đó, predicate pushdown giúp đẩy điều kiện filter xuống gần bước scan dữ liệu để giảm số row cần xử lý, còn projection pushdown giúp chỉ đọc các column thật sự cần thiết. Nhờ vậy, Polars có thể giảm I/O, giảm memory usage và hạn chế intermediate materialization trong các pipeline nhiều bước. Đây là lý do Polars thường mạnh ở các workload như filter, groupby và analytical pipeline.

Tuy nhiên, Polars chủ yếu tối ưu cho single-machine analytics. Nếu bài toán yêu cầu distributed execution trên nhiều worker hoặc cluster, Polars không phải lựa chọn trực tiếp như Dask. Vì vậy, Polars phù hợp nhất khi dữ liệu lớn nhưng vẫn có thể xử lý hiệu quả trên một máy đủ mạnh.

4.3 Dask Architecture

Dask có kiến trúc dựa trên partitioning và task graph. Với Dask DataFrame, một dataset lớn được chia thành nhiều partition, và mỗi partition có thể được xử lý như một DataFrame nhỏ hơn. Khi người dùng gọi operation trên Dask DataFrame, Dask không thực thi ngay mà tạo task graph mô tả các task cần chạy và quan hệ phụ thuộc giữa chúng. Chỉ khi gọi `compute()`, scheduler mới bắt đầu thực thi các task trên local cores hoặc distributed workers.



Hình 3: Simplified Dask architecture

Kỹ thuật cốt lõi của Dask là chia computation thành nhiều task nhỏ và thực thi chúng song song. Partitioning giúp Dask xử lý từng phần dữ liệu độc lập, task graph cho phép biểu diễn dependency giữa các bước xử lý, còn scheduler chịu trách nhiệm quyết định thứ tự chạy và phân phối task cho worker. Nhờ mô hình này, Dask có thể chạy trên một máy nhiều cores hoặc mở rộng sang distributed cluster.

Dask phù hợp với các workload có thể chia nhỏ tốt, chẳng hạn đọc nhiều file độc lập, xử lý từng partition, aggregation theo partition hoặc pipeline có thể parallelize. Trong một số trường hợp, Dask có thể xử lý workload lớn hơn RAM nhờ partitioned execution và out-of-core style processing. Tuy nhiên, Dask cũng có overhead từ task graph, scheduler và shuffle. Các thao tác như join lớn, groupby với key bị skew hoặc workload cần data movement nhiều có thể làm Dask chậm hơn các engine single-machine tối ưu như Polars.

5 Dataset and Benchmark Workloads

5.1 Dataset Overview

Báo cáo sử dụng dataset Amazon Reviews 2023, cụ thể là category *Clothing, Shoes and Jewelry*. Đây là tập dữ liệu review sản phẩm có quy mô lớn, phù hợp để đánh giá các framework DataFrame trên dữ liệu thực. Trong benchmark chính, real dataset được chia thành các mốc 1M, 10M và 50M rows để kiểm tra khả năng scale theo *row count*.

Ngoài real data, báo cáo cũng sử dụng synthetic data cho các tình huống stress test. Synthetic data không thay thế real data, mà dùng để tạo các trường hợp khó kiểm soát trong dữ liệu thật, chẳng hạn 100M rows, dữ liệu bị skew mạnh hoặc dữ liệu có nhiều unique ID. Cách tách real data và synthetic data giúp benchmark vừa phản ánh workload thực tế, vừa kiểm tra được giới hạn của từng framework.

Một điểm quan trọng là *row count* và *physical dataset size* không giống nhau. Hai dataset có cùng số dòng vẫn có thể có kích thước file khác nhau do độ dài text, compression ratio hoặc số lượng unique values. Vì vậy, benchmark tách riêng row scaling và physical scaling để tránh diễn giải sai kết quả.

5.2 Benchmark Workloads

Benchmark sử dụng bốn workload chính: filter, groupby, join và pipeline. Đây là các thao tác phổ biến trong phân tích dữ liệu dạng bảng và đủ để thể hiện sự khác biệt giữa Pandas, Polars và Dask.

Filter workload kiểm tra khả năng đọc dữ liệu, áp dụng điều kiện lọc và materialize kết quả. Workload này thường hưởng lợi từ các kỹ thuật như *predicate pushdown* và *projection pushdown*, đặc biệt trong Polars Lazy. Groupby workload kiểm tra khả năng aggregation trên dữ liệu lớn. Đây là workload nhạy với key cardinality và skewness, vì số lượng group hoặc phân phối key có thể ảnh hưởng lớn đến hash table, memory usage và partition balance.

Join workload đánh giá khả năng kết hợp hai bảng dựa trên key. Đây thường là workload nặng về memory vì có thể tạo intermediate data và output lớn. Với Dask, join còn có thể phát sinh shuffle giữa các partition. Cuối cùng, pipeline workload kết hợp nhiều bước như filter, select, transform và groupby. Workload này gần với thực tế hơn vì mô phỏng một chuỗi xử lý

dữ liệu hoàn chỉnh, đồng thời làm rõ sự khác biệt giữa eager execution và lazy execution.

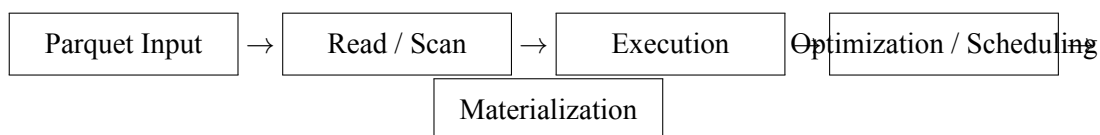
Bảng 2: Summary of benchmark workloads

Workload	Purpose	Main factors
Filter	Đánh giá scan, filtering và result materialization.	Predicate pushdown, projection pushdown, output size.
GroupBy	Đánh giá aggregation trên dữ liệu lớn.	Key cardinality, skewness, hash table size.
Join	Đánh giá khả năng kết hợp bảng theo key.	Join key size, shuffle, memory pressure, output size.
Pipeline	Đánh giá workflow nhiều bước gần với thực tế.	Query optimization, task graph, intermediate materialization.

6 Benchmark Methodology

6.1 Benchmark Objective

Mục tiêu của benchmark là so sánh hiệu năng thực tế của Pandas, Polars và Dask trong các workload DataFrame phổ biến. Benchmark không đo tốc độ thuần túy của một operator riêng lẻ, mà đo *end-to-end workload performance*. Điều này có nghĩa là runtime bao gồm toàn bộ quá trình từ đọc dữ liệu Parquet, thực thi operation, tối ưu hoặc lập lịch, cấp phát bộ nhớ cho đến materialize kết quả cuối cùng.



Hình 4: End-to-end benchmark execution path

Cách đo này phản ánh thực tế sử dụng tốt hơn so với việc chỉ đo riêng một operator. Khi làm việc với dữ liệu lớn, người dùng không chỉ quan tâm một phép tính chạy nhanh đến đâu, mà còn quan tâm framework có đọc được dữ liệu, xử lý ổn định và tạo ra kết quả cuối cùng hay không.

6.2 Execution Strategy

Để đảm bảo các framework thực sự thực thi workload, benchmark ép materialization ở cuối mỗi lần chạy. Pandas vốn dùng eager execution nên các thao tác thường được chạy ngay. Với Polars

Lazy, benchmark gọi `collect()` để thực thi query plan và lấy kết quả. Với Dask, benchmark gọi `compute()` để thực thi task graph và materialize output.

Mỗi cấu hình benchmark được chạy với một warmup run và ba lần repeat chính. Warmup run được loại bỏ khỏi kết quả để giảm nhiễu ban đầu. Các lần repeat chính được dùng để tính runtime trung bình, peak memory và throughput.

6.3 Benchmark Groups

Benchmark được chia thành bốn nhóm để mỗi nhóm kiểm soát một yếu tố chính. Nhóm đầu tiên là *Real Data Row Scaling*, sử dụng real data với các mốc 1M, 10M và 50M rows nhằm đánh giá khả năng scale theo số dòng. Nhóm thứ hai là *Physical Scaling and Memory Pressure*, tập trung vào kích thước vật lý của dataset như 5GB, 10GB và 20GB để xem framework phản ứng thế nào khi memory pressure tăng.

Nhóm thứ ba là *Synthetic Stress Benchmark*, sử dụng synthetic data để kiểm tra các tình huống cực đoan như 100M rows, heavy skewness và high-cardinality IDs. Nhóm cuối cùng là *Environment Comparison*, so sánh kết quả giữa các môi trường như Windows local và Colab Linux. Phần này không được xem là so sánh hệ điều hành tuyệt đối, vì các môi trường còn khác nhau về hardware, RAM, filesystem và runtime policy.

6.4 Metrics and Interpretation

Metric chính của benchmark là *runtime*. Ngoài ra, benchmark cũng ghi nhận *peak memory*, throughput, dataset size, row count, output size và success/failure status nếu có. Các metric phụ này giúp giải thích tại sao một workload nhanh hoặc chậm, đặc biệt trong các trường hợp memory pressure hoặc output materialization lớn.

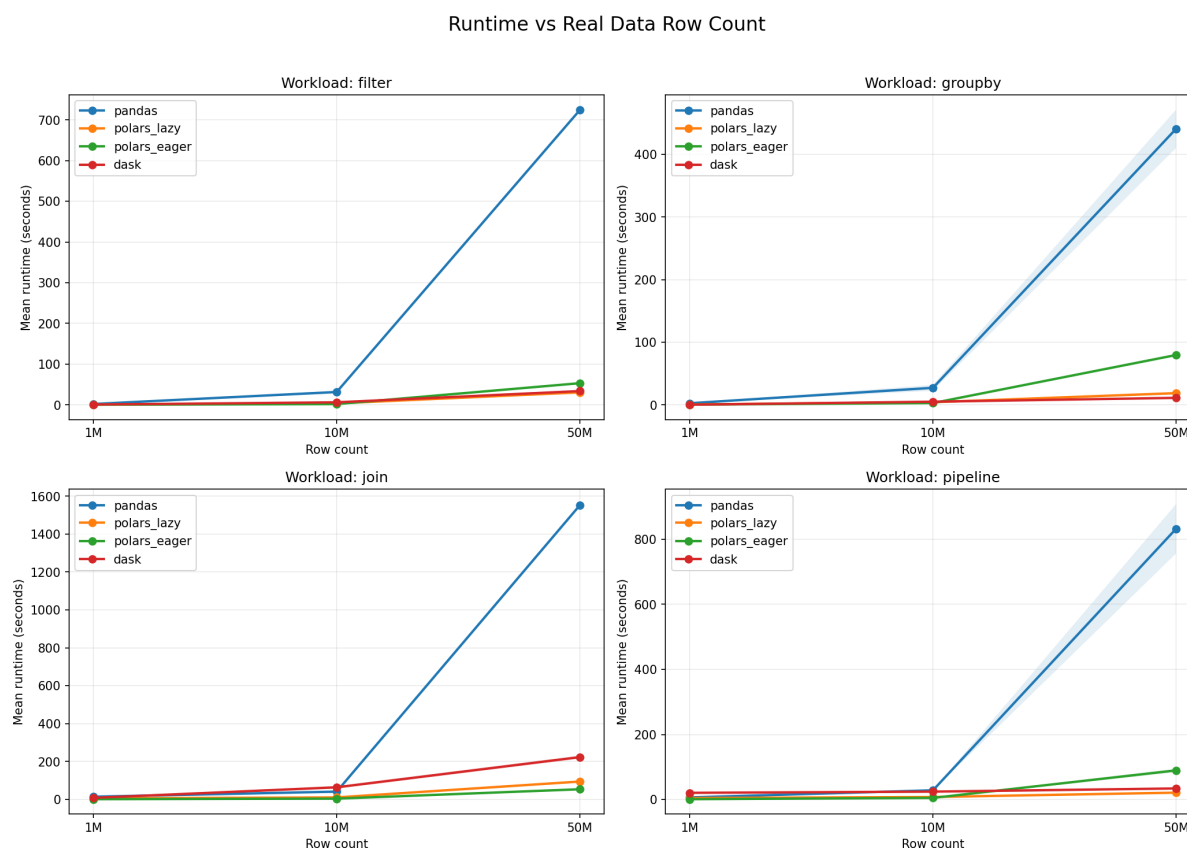
Khi diễn giải kết quả, cần nhớ rằng benchmark này là end-to-end benchmark. Do đó, không nên hiểu runtime là pure engine time. Ví dụ, filter workload không chỉ đo chi phí evaluate điều kiện lọc, mà còn bao gồm đọc Parquet, decode dữ liệu, cấp phát memory và tạo output. Tương tự, join workload không chỉ đo thuật toán join, mà còn bao gồm key handling, memory allocation, shuffle nếu có và materialization.

Vì vậy, kết quả benchmark nên được hiểu là hiệu năng thực tế của framework trong một workflow hoàn chỉnh. Đây là cách đo phù hợp với mục tiêu của báo cáo: so sánh Pandas, Polars và Dask trong điều kiện sử dụng gần với thực tế.

7 Experimental Results and Analysis

7.1 Real Data Row Scaling

Thí nghiệm này đánh giá khả năng scale khi real dataset tăng từ 1M lên 10M và 50M rows trên bốn workload: filter, groupby, join và pipeline.



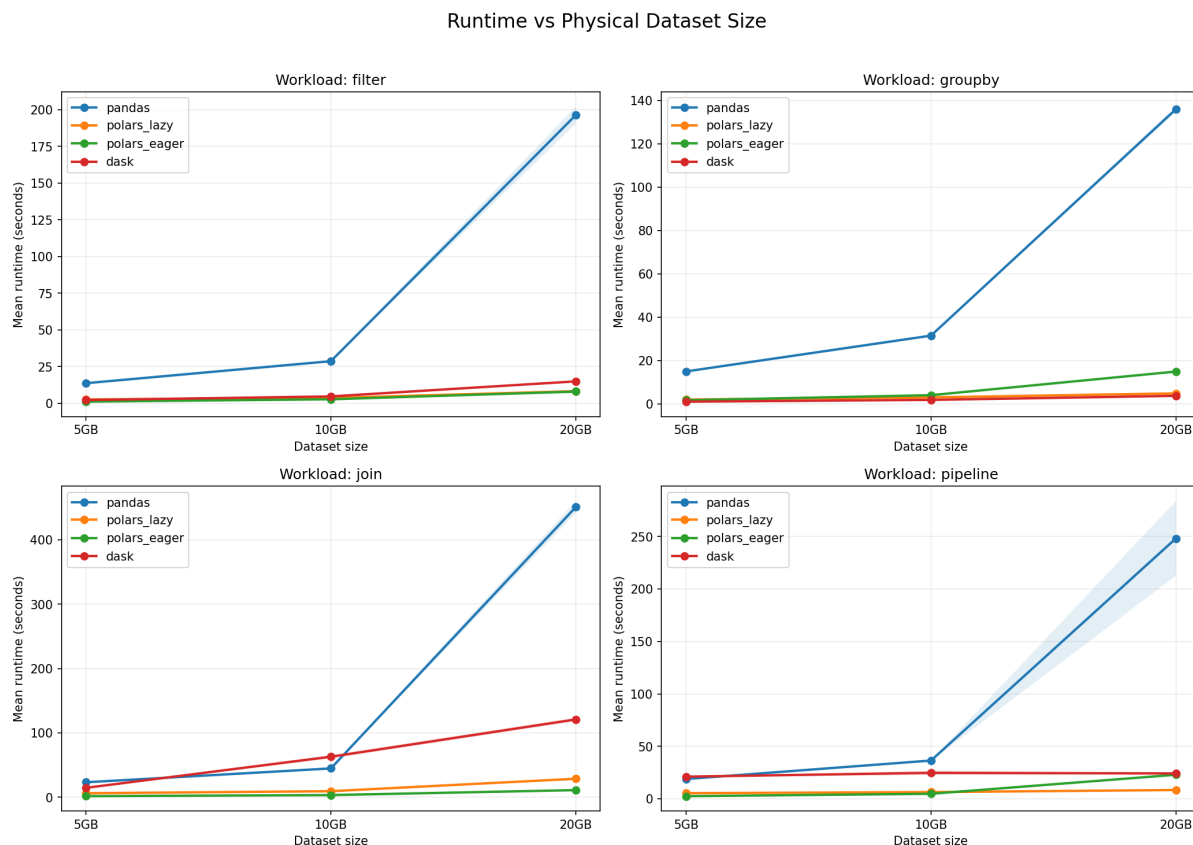
Hình 5: Runtime theo real data row count trên các workload

Figure 5 cho thấy Pandas scale yếu nhất. Khi dữ liệu tăng lên 50M rows, runtime của Pandas tăng mạnh, đặc biệt ở filter, join và pipeline. Polars nhanh hơn rõ rệt ở hầu hết workload lớn. Polars Lazy có lợi thế ở filter và pipeline nhờ query optimization, trong khi Polars Eager nổi bật ở join. Dask cũng nhanh hơn Pandas khi dữ liệu lớn, nhưng không luôn vượt Polars do overhead từ task graph, scheduler và partition management.

Nhìn chung, kết quả này cho thấy Pandas phù hợp làm baseline, còn Polars và Dask phù hợp hơn cho dataframe workload lớn. Polars mạnh về single-machine performance, trong khi Dask hữu ích khi workload có thể chia partition hiệu quả.

7.2 Physical Scaling and Memory Pressure

Thí nghiệm này đánh giá runtime khi physical dataset size tăng, ví dụ 5GB, 10GB và 20GB. Khác với row scaling, physical scaling tập trung vào kích thước dữ liệu thực trên đĩa và áp lực bộ nhớ.

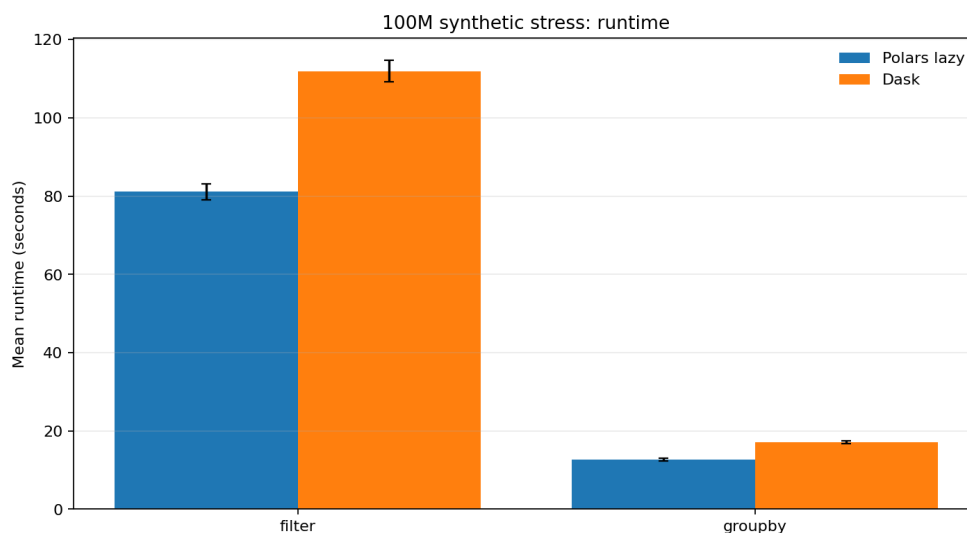


Hình 6: Runtime theo physical dataset size trên các workload

Figure 6 cho thấy Pandas bị ảnh hưởng mạnh khi dataset size tăng và trở nên chậm nhất ở mức 20GB. Polars duy trì runtime thấp trong hầu hết trường hợp; Polars Lazy mạnh ở filter, groupby và pipeline, còn Polars Eager tiếp tục tốt ở join. Dask nhanh hơn Pandas nhưng kém ổn định hơn Polars, nhất là với join do có thể phát sinh shuffle và partition coordination.

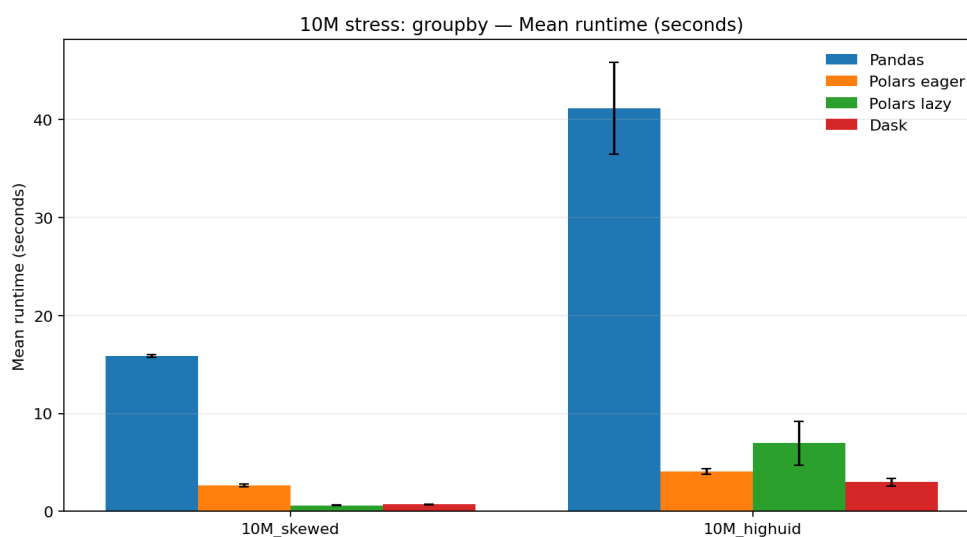
7.3 Synthetic Stress Benchmark

Synthetic stress benchmark kiểm tra các tình huống khó như 100M rows, skewed keys và high-cardinality identifiers. Các kết quả này dùng để đánh giá robustness, không thay thế real-data benchmark.



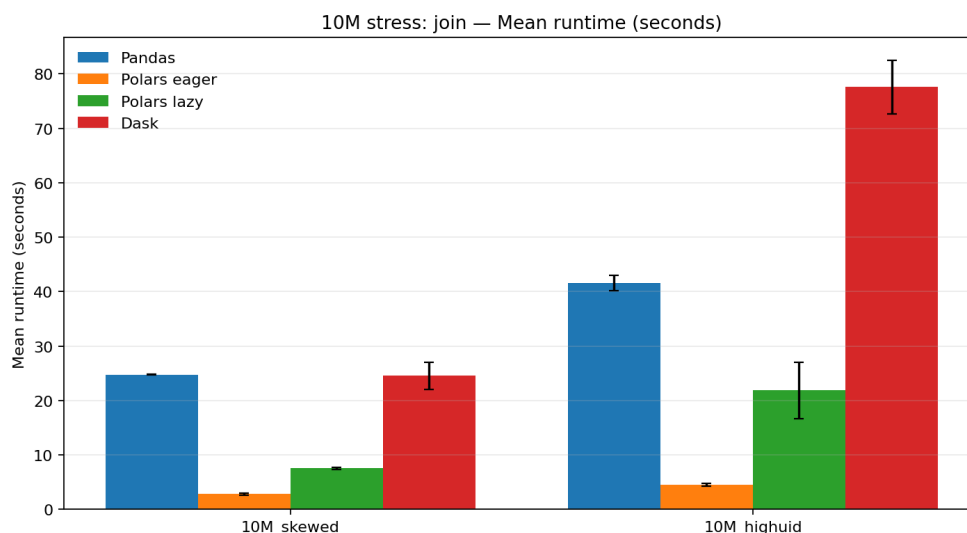
Hình 7: Runtime trên 100M synthetic stress benchmark

Figure 7 cho thấy Polars Lazy nhanh hơn Dask ở cả filter và groupby trên synthetic dataset 100M rows. Điều này cho thấy Polars Lazy tận dụng tốt query optimization và columnar execution trên single machine. Dask vẫn hoàn thành workload, nhưng overhead từ scheduler và partition management làm runtime cao hơn trong thí nghiệm này.



Hình 8: Runtime cho 10M stress groupby workloads

Figure 8 cho thấy Pandas chậm nhất trong các stress groupby workload. Polars Lazy nhanh nhất ở skewed groupby, trong khi Dask đạt kết quả tốt ở high-UID groupby. Điều này cho thấy framework tốt nhất có thể thay đổi theo data distribution.

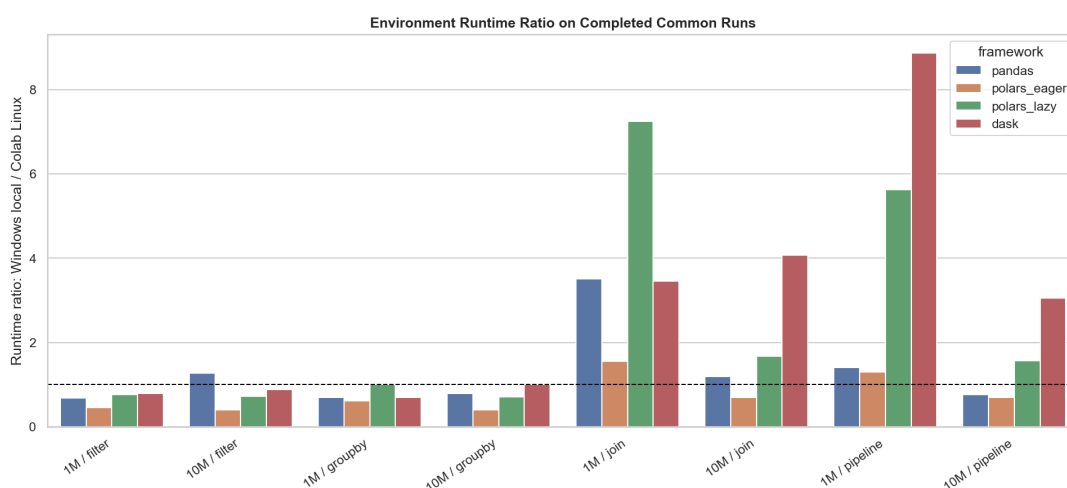


Hình 9: Runtime cho 10M stress join workloads

Figure 9 cho thấy Polars Eager là framework nhanh nhất cho stress join. Dask có runtime cao, đặc biệt ở high-UID case, vì join có thể cần shuffle hoặc data movement lớn. Kết quả này cho thấy partitioned execution không tự động đảm bảo hiệu năng tốt cho join-heavy workload.

7.4 Environment Comparison

Environment comparison so sánh local Windows environment và Colab Linux environment. Đây không phải pure OS comparison vì hai môi trường khác nhau về hardware, RAM, filesystem và runtime policy.

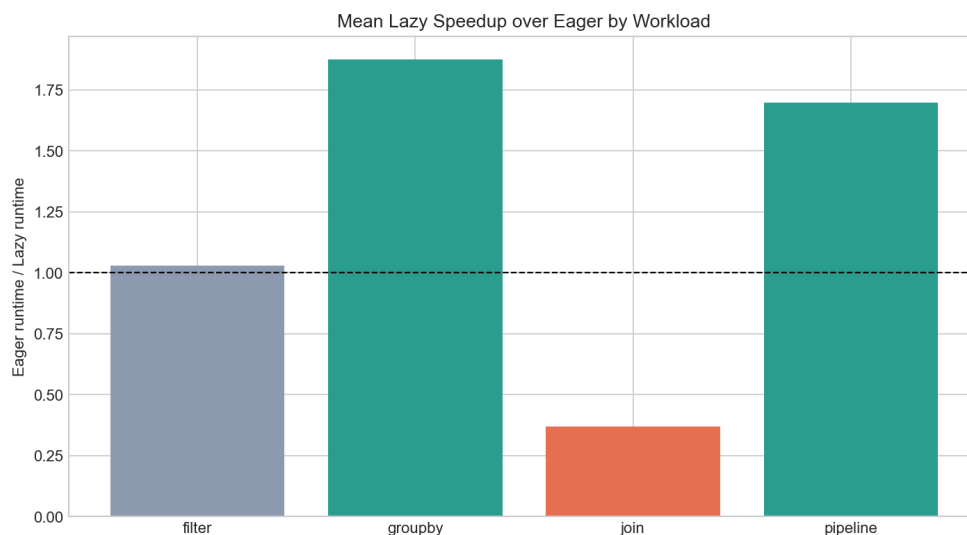


Hình 10: Runtime ratio giữa Windows local và Colab Linux trên completed common runs

Figure 10 cho thấy runtime thay đổi đáng kể theo môi trường. Dask là framework nhạy nhất với environment, đặc biệt ở join và pipeline. Polars Eager có xu hướng ổn định hơn. Vì vậy, kết quả benchmark cần được diễn giải trong bối cảnh môi trường chạy cụ thể.

7.5 Polars Lazy vs Eager

Polars hỗ trợ cả Lazy và Eager execution. Kết quả benchmark cho thấy Lazy không luôn nhanh hơn Eager, mà phụ thuộc vào workload.



Hình 11: Mean Lazy speedup over Eager theo workload

Figure 11 cho thấy Lazy nhanh hơn ở groupby và pipeline, vì các workload này hưởng lợi từ query optimization và giảm intermediate materialization. Với filter, Lazy và Eager gần tương đương. Ngược lại, với join, Polars Eager nhanh hơn rõ rệt. Vì vậy, Lazy phù hợp hơn với pipeline hoặc aggregation, còn Eager phù hợp hơn với join-heavy workload.

8 Multi-aspect Comparison

Từ kiến trúc và kết quả benchmark, có thể thấy Pandas, Polars và Dask phục vụ các mục tiêu khác nhau dù đều cung cấp DataFrame API. Pandas mạnh về sự đơn giản và tính phổ biến, phù hợp cho exploratory data analysis, data cleaning và prototyping trên dữ liệu nhỏ đến vừa. Tuy nhiên, Pandas không phải là Big Data engine, nên dễ gặp giới hạn khi dữ liệu lớn hơn RAM hoặc pipeline tạo nhiều intermediate DataFrame.

Polars tập trung vào high-performance single-machine analytics. Nhờ Rust execution engine, columnar memory layout, multi-threading và query optimization, Polars thường có runtime tốt trong các workload lớn trên một máy đơn. Polars Lazy phù hợp với groupby và pipeline vì có thể tối ưu query plan, trong khi Polars Eager thường mạnh ở các thao tác trực tiếp như join.

Dask tập trung vào scalability. Thay vì tối ưu toàn bộ engine trên một máy đơn, Dask chia dữ liệu thành nhiều partition, xây dựng task graph và dùng scheduler để thực thi song song hoặc phân tán. Vì vậy, Dask phù hợp khi dữ liệu lớn hơn memory của một máy, workload có

thể partition tốt, hoặc khi cần mở rộng sang distributed cluster. Tuy nhiên, Dask có thể chịu overhead từ scheduler, task graph và shuffle, đặc biệt trong join-heavy workload.

Bảng 3: Multi-aspect comparison of Pandas, Polars, and Dask

Aspect	Pandas	Polars	Dask
Main goal	Simple data analysis	Fast single-machine analytics	Parallel/distributed analytics
Execution model	Eager execution	Eager and Lazy execution	Lazy task graph execution
Strength	Easy to use, popular ecosystem	Fast runtime, query optimization	Partitioning and scalability
Best workloads	Small/medium EDA	Filter, join, groupby, pipeline	Partition-friendly groupby and large workflows
Main weakness	RAM bottleneck	Not primarily distributed	Scheduler and shuffle overhead

Nhìn chung, Pandas phù hợp nhất với dữ liệu nhỏ đến vừa và vai trò baseline. Polars là lựa chọn mạnh khi cần xử lý nhanh trên single machine. Dask phù hợp hơn khi dữ liệu hoặc workload cần được chia partition và có khả năng mở rộng sang môi trường distributed.

9 Feature Demonstration and Scenario-based Comparison

Phần này minh họa cách Pandas, Polars và Dask khác nhau trong các workload phổ biến như filter, groupby, join và pipeline. Mục tiêu không phải là trình bày cách cài đặt, mà là làm rõ khi nào mỗi framework có lợi thế.

Với filter workload, Pandas dễ dùng nhưng phải đọc và xử lý dữ liệu trong memory, nên dễ gặp giới hạn khi dataset lớn. Polars Lazy có lợi thế nhờ *predicate pushdown* và *projection pushdown*, giúp giảm lượng dữ liệu cần scan. Dask có thể filter theo từng partition, phù hợp với dữ liệu đã được chia nhỏ, nhưng vẫn có thể chịu memory pressure khi gọi `compute()` nếu output lớn.

Với groupby workload, Pandas phù hợp ở scale nhỏ nhưng chậm hơn khi số row hoặc số group tăng. Polars thường nhanh nhờ execution engine tối ưu và multi-threading; riêng Polars Lazy có thể tối ưu query plan trước khi chạy. Dask có thể cạnh tranh tốt nếu aggregation được thực hiện hiệu quả theo partition, nhưng sẽ bị ảnh hưởng nếu key bị skew hoặc cần shuffle nhiều.

Với join workload, Polars Eager là lựa chọn nổi bật trong benchmark này. Join thường gây áp lực memory vì cần xử lý join key, hash table và output lớn. Dask có thể gặp overhead

đáng kể nếu join yêu cầu shuffle hoặc repartitioning, trong khi Pandas dễ bị giới hạn bởi RAM khi input hoặc output lớn.

Với pipeline workload, Polars Lazy thể hiện rõ lợi thế vì toàn bộ chuỗi thao tác được đưa vào query plan trước khi execution. Điều này giúp optimizer giảm intermediate materialization và đẩy filter/projection lên sớm. Dask cũng phù hợp nếu pipeline có thể chia partition tốt, còn Pandas phù hợp hơn cho prototype hoặc pipeline nhỏ.

Bảng 4: Scenario-based recommendation

Scenario	Recommended Tool	Reason
Small exploratory analysis	Pandas	Simple API and rich ecosystem
Large single-machine analytics	Polars	Fast columnar execution
Multi-step pipeline	Polars Lazy	Query optimization reduces intermediate data
Join-heavy workload	Polars Eager	Strong join performance in benchmark
Partitioned or larger-than-memory data	Dask	Partitioned and scalable execution

10 Limitations

Benchmark này có một số hạn chế cần lưu ý. Thứ nhất, đây là *end-to-end benchmark*, không phải benchmark đo riêng tốc độ nội bộ của engine. Runtime bao gồm đọc Parquet, decode dữ liệu, execution, optimization hoặc scheduling, memory allocation và final result materialization. Vì vậy, kết quả nên được hiểu là hiệu năng thực tế của workflow hoàn chỉnh, không phải pure operator speed.

Thứ hai, kết quả phụ thuộc vào hardware và environment. CPU, RAM, storage, filesystem, OS memory management, pagefile, swap và runtime limit đều có thể ảnh hưởng đến runtime và peak memory. Do đó, so sánh giữa Windows local và Colab Linux nên được xem là *environment comparison*, không phải pure OS comparison.

Thứ ba, việc ép framework tạo output cuối cùng bằng Pandas eager execution, Polars `collect()` và Dask `compute()` làm kết quả phụ thuộc vào output size. Một groupby có thể có input lớn nhưng output nhỏ, trong khi filter hoặc join có thể tạo output lớn và gây memory pressure.

Cuối cùng, synthetic data chỉ dùng để kiểm tra stress case như 100M rows, skewness hoặc high-cardinality IDs. Các kết quả này giúp phân tích robustness, nhưng không thay thế hoàn toàn real data benchmark. Ngoài ra, Dask trong báo cáo chủ yếu được đánh giá trong môi trường local/Colab, nên chưa phản ánh đầy đủ khả năng distributed cluster trong production.

11 Conclusion and Recommendations

Pandas được dùng làm baseline vì phổ biến và dễ dùng, nhưng kết quả cho thấy Pandas không phù hợp nhất khi dữ liệu tăng lớn. Polars là framework có hiệu năng single-machine tốt nhất trong nhiều thí nghiệm. Polars Lazy phù hợp với groupby và pipeline nhờ query optimization, predicate pushdown và giảm intermediate materialization. Dask phù hợp với các workload có thể tận dụng partitioned execution hoặc cần mở rộng sang nhiều core/worker. Tuy nhiên, Dask có thể chịu overhead từ scheduler, task graph, partition management và shuffle, đặc biệt trong join-heavy hoặc high-cardinality cases.

Từ các kết quả trên, có thể rút ra khuyến nghị chính như sau: Pandas phù hợp cho dữ liệu nhỏ, exploratory analysis và prototyping; Polars phù hợp cho xử lý nhanh trên single machine; Polars Lazy nên được ưu tiên cho pipeline và aggregation; Polars Eager phù hợp với join-heavy workload; Dask phù hợp khi dữ liệu hoặc workload cần partitioning hoặc có khả năng mở rộng sang distributed execution. Kết luận cuối cùng là không có framework nào tốt nhất trong mọi trường hợp. Lựa chọn công cụ nên dựa trên workload type, data size, memory pressure, output size và execution environment.

References

Tài liệu

- [1] Pandas Development Team. *Pandas Documentation and Project Information*. Available at: <https://pandas.pydata.org/>
- [2] Pandas Development Team. *Pandas License*. Available at: <https://github.com/pandas-dev/pandas/blob/main/LICENSE>
- [3] Polars. *Polars: DataFrames for the New Era*. Available at: <https://pola.rs/>
- [4] Polars Developers. *Polars License*. Available at: <https://github.com/pola-rs/polars/blob/main/LICENSE>
- [5] Dask Development Team. *Dask Documentation*. Available at: <https://docs.dask.org/>
- [6] Dask Developers. *Dask License*. Available at: <https://github.com/dask/dask/blob/main/LICENSE.txt>
- [7] McAuley Lab. *Amazon Reviews 2023 Dataset*. Available at: <https://amazon-reviews-2023.github.io/>
- [8] McAuley Lab. *Amazon-Reviews-2023 on Hugging Face*. Available at: <https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023>